

EvolveSWDB weekly update

Profiling a workload and rewriting its code, with BFS as the first case

Yan-Ru Jhou 2026-09-30

Where we are

Last week

Built the Software Database

- Real code + memory access patterns
- Inputs, machine, measured profiles
- Queries and exports for agents

This week: my part

Make the database actionable

- Profiling: where does time go?
- Rewriting: proposal → real code
- Check + measure every candidate

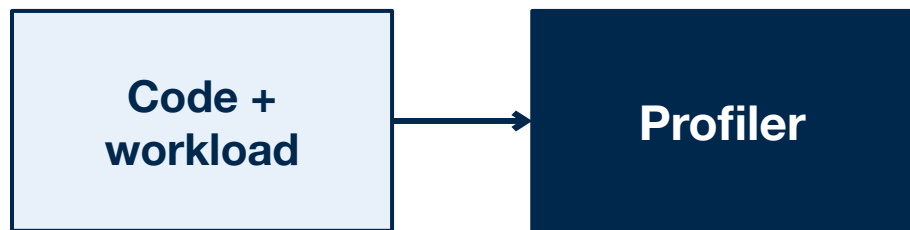
First case: BFS. DX100 is one prototype evaluation target.

Why profiling and rewriting?

	Profiling	Rewriting
Problem	Which code is hot on this input?	A proposal is not yet code
Problem	A rewrite adds code no catalog knows	Edits can drift out of scope
Solution	Discover functions + loops, attach memory data	Apply to exact source, bounded repair
Result	Profile package for the agent	Checked, measured candidate

The agent decides what to change; we supply evidence and a trustworthy verdict.

Profiling: what the agent receives



Output: one profile package

Hot functions + loops

found by parsing source; no annotations

CPU time per region

inclusive and exclusive kept apart

Memory behavior

reads, writes, modeled cache misses

Context + strategies

source, callers, matching strategies

No hardware counters on the shared host: memory data is simulated and labeled so.

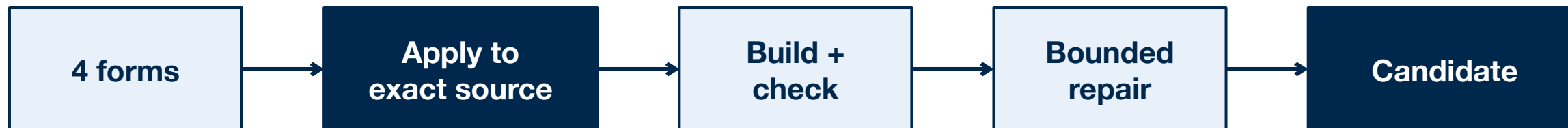
Example: a rewrite adds a new hot helper

	Before rewrite	After rewrite
Functions found	11	12 (new helper ranked #1)
Loops found	20	22 (2 new nested loops)
Unchanged fragments matched	—	30
Data reads, source 0 (Callgrind)	8,468	54,474

Diagnostic graph: 10 vertices, sources 0, 3, 8, 1 thread. Checks tracking, not speed.

Profiling follows the code after a rewrite; old numbers never carry over to changed code.

Rewriting: four proposal forms, one path



Form	Real example	Outcome
Annotated source	GAPBS BFS: tuning parameter alpha 15 → 16	Correct
Structured instructions	GAPBS BFS: remove one dead reset call	Correct
Patch	DX100 scalar BFS: 64-vertex chunks, drop redundant store	Correct

Repairs fix only build or correctness failures; they never tune performance.

Measuring the rewrites: no change yet

mbit10 Xeon Gold 6326, 1 thread, 10 paired repetitions, BFS call only

Starting code + form	Graph	Baseline (ms)	Candidate (ms)	Speedup	95% CI
DX100 scalar BFS, patch	Uniform	46.03	45.96	1.002	[0.998, 1.011]
DX100 scalar BFS, patch	Kronecker	23.26	23.23	1.000	[0.996, 1.020]
GAPBS BFS, instructions	Uniform	7.33	7.29	1.002	[0.999, 1.005]
GAPBS BFS, instructions	Kronecker	6.00	5.99	1.001	[0.999, 1.030]

Correct on all timed trials. Spread above the 10% limit, so the verdict is “inconclusive”.

Small edits, no gain. The path from proposal to verdict works end to end.

Ongoing work and what we learned

Area	Limitation	Response
Rewriting	LLM timeouts, one patch failed to apply	Whole-file edit format; new candidate
Profiling	Headers, libraries, 1 OpenMP region not attributed	Reported as partial coverage
DX100 prototype	gem5 runs take hours; 2 jobs per host	Parallel runs within one socket
Harness	Save cost per trial	8.7 s → 1.0 s

Lesson: report what is not covered instead of hiding it.

Plan for this week

1

Hand off annotated BFS to Peter's agent

Pinned source, build and graph commands, logs, Josh's 7 statement IDs

2

Offer our gem5 DX100 path as the evaluator

Answers “who builds and tests the proposed hardware?”

3

Then: profile-driven rewrite on a pilot-sized graph

Target a measured hot loop, not a toy edit

Team pipeline (09-24): Yan-Ru → Peter → Josh/Eric → Peter → Yan-Ru

Backup: native measurement protocol

Setting	Value
Host	mbit10, Intel Xeon Gold 6326, one socket lane
Threads	1
Repetitions	10 paired (baseline / candidate alternating)
Claim rule	Speedup ≥ 1.05 and spread $\leq 10\%$
Correctness	BFS tree checked on every timed trial
Profiling	Region timers + Callgrind (no HW counters)